

Project Hermes

An autonomous library rover using an offboard PC 'Brain' (Python/CV/LLM) to power an ESP32 'Body' for advanced stereo vision navigation and conversational AI.

Rohit K. S., Hiramoni K.

1. Core Philosophy & Team Roles

This project is built on a distributed **"Offboard Brain / Onboard Body"** architecture. This design philosophy is a deliberate engineering choice that separates complex, resource-heavy computation from the real-time physical control of the robot. This separation is what allows us to build a highly intelligent robot using simple, cost-effective hardware.

Here is a detailed breakdown of this two-part system:

1. The "Offboard Brain"

The "Brain", which runs all the high-level, computationally-expensive software. It is the robot's consciousness, handling all perception, decision-making, and conversational intelligence.

- **Role:** To perceive the world, think, and decide what to do.
- **Key Responsibilities:**
 - **Heavy Computation:** It leverages your powerful CPU and, most critically, your **CUDA-enabled GPU (GTX 1650)**.
 - **Conversational AI:** It runs the entire Speech-to-Speech (S2S) pipeline, including the large language model (LLM) for processing and generating natural language responses.
 - **Stereo Vision:** It receives two simultaneous video streams from the robot and performs the computationally intensive stereo-matching algorithms to create a 3D depth map of the environment.
 - **Perception & Navigation:** It runs the AprilTag detection algorithms to determine the robot's location and the obstacle detection logic on the 3D depth map.

- **Pathfinding:** It hosts the "map" of the library (as a networkx graph) and runs Dijkstra's algorithm to calculate the optimal path—and dynamically recalculate a new one when an obstacle is detected.

2. The "Onboard Body" (The ESP32 Robot)

The "Body" is the physical robot chassis. Its only job is to be a reliable, real-time executor of simple commands and a data-forwarding service for the "Brain."

- **Role:** To execute physical commands *immediately* and stream sensor data.
- **Location:** The 4WD chassis with its ESP32 controllers.
- **Key Responsibilities:**
 - **Low-Latency Execution:** The ESP32 firmware (written in C++) is excellent at real-time tasks. When it receives a command like {"direction": "forward"}, it executes it instantly.
 - **Precise Motor Control:** It handles all the high-frequency tasks of driving the four motors via the DRV8833 drivers.
 - **Sensor Reading:** It continuously counts pulses from the four wheel encoders and reads data from the MPU-6050 IMU. This is a real-time task that cannot be interrupted by a slow-running OS.
 - **Data Funnel:** It acts as a bridge for the I/O. It streams raw audio from the INMP441 mic *to* the Brain and plays audio received *from* the Brain on its speaker. The ESP32-CAMs stream video directly *to* the Brain.

3. The "Nervous System" (The MQTT & Wi-Fi Seam)

The "Brain" and "Body" are decoupled and communicate wirelessly over a "Nervous System" built on Wi-Fi. The **MQTT protocol** acts as the formal "API contract" between them.

- The **Brain** publishes commands to topics like robot/commands/move.
- The **Body** subscribes to these topics and executes them, no questions asked.
- The **Body** publishes raw data to topics like robot/audio/instream or robot/status/location.
- The **Brain** subscribes to this data to make its decisions.

Lead Software Engineer (PC "Brain")

Responsibilities: All Python code running on the host PC.

- NLP/LLM Speech Pipeline
- Computer Vision (Stereo, AprilTags)
- Navigation & Pathfinding (NetworkX)
- Master Control Script (`main\_controller.py`)
- Publishing commands via MQTT

Lead Firmware Engineer (Robot "Body")

Responsibilities: All C++/Arduino code running on the ESP32s.

- Hardware Integration & Wiring
- Motor Control (DRV8833)
- Sensor Reading (Encoders, IMU)
- User I/O (Mic, OLED, Speaker)
- Subscribing to commands via MQTT

| 2. The "Seam": MQTT API Contract

This API is the **Single Source of Truth** for all communication. All messages ****MUST**** adhere to this format. The "Brain" and "Body" do not talk to each other; they talk to these topics.

PC (Brain) -> Robot (Body)

Topics the Robot's main ESP32 ****MUST**** subscribe to.

`robot/commands/move`

Description: Commands the robot to drive the motors.

Message (JSON): ``{"direction": "forward", "speed_percent": 80}``

Valid `direction` values: "forward", "backward", "stop"

`robot/commands/turn`

Description: Commands the robot to turn a specific amount.

Message (JSON): ``{"direction": "left", "degrees": 90}``

Valid `direction` values: "left", "right"

`robot/commands/display`

Description: Updates the text on the robot's OLED screen.

Message (JSON): ``{"line1": "Navigating...", "line2": "To: History"}``

`robot/audio/out_stream`

Description: A stream of raw audio chunks for the robot to play on its speaker.

Message (Raw Bytes): ``[audio_chunk_data]``

Robot (Body) -> PC (Brain)

Topics the PC's Python scripts ****MUST**** subscribe to.

`robot/status/location`

Description: Sent by the PC's vision system when a new AprilTag is confidently identified.

Message (JSON): ``{"current_tag_id": 5, "timestamp": 1678886400}``

`robot/events/obstacle`

Description: Sent by the PC's vision system when the stereo camera detects a blockage.

Message (JSON): ``{"status": "blocked", "distance_cm": 30}``

`robot/status/imu`

Description: Stream of IMU data for sensor fusion (optional).

Message (JSON): ``{"yaw": 90.1, "pitch": 0.5, "roll": -0.2}``

`robot/audio/in_stream`

Description: A stream of raw audio chunks from the robot's microphone.

Message (Raw Bytes): ``[audio_chunk_data]``

`robot/video/{left_cam_id}/stream_url`

Description: Sent by the LEFT ESP32-CAM on startup. ``{left_cam_id}`` is its unique ID.

Message (String): ``http://192.168.1.105:81/stream``

robot/video/{right_cam_id}/stream_url

Description: Sent by the RIGHT ESP32-CAM on startup. `{right\_cam\_id}` is its unique ID.

Message (String): `http://192.168.1.106:81/stream`

| 3. GitHub Repository Structure

Our GitHub repository is a "monorepo" that holds all project files. This keeps all code and documentation in one place.

```
/project-hermes/
├── .gitignore      (Ignores temp files)
├── README.md       (Project front page - with GIF/video)
├── docs/           (All project documentation)
│   ├── MQTT_API.md (A markdown version of the API contract)
│   └── setup_guide.md (Installation instructions)
├── hardware/       (3D models, wiring diagrams)
│   ├── wiring_diagram.png
│   └── stl/
│       └── camera_mount.stl
├── pc_brain/       (All Python "Brain" code)
│   ├── cv/
│   │   ├── stereo_vision.py
│   │   └── apriltag_detector.py
│   ├── nlp/
│   │   └── speech_pipeline.py
│   ├── navigation/
│   │   └── path_planner.py
│   ├── main_controller.py (Main PC script)
│   └── requirements.txt   (Python libraries)
└── robot_body/     (All C++ "Body" firmware)
    ├── esp32_main_controller/ (PlatformIO project for 38-pin ESP32)
    │   ├── platformio.ini
    │   ├── src/
    │   │   └── main.cpp
    │   └── lib/
    ├── esp32_cam_left/    (PlatformIO project for left camera)
    └── esp32_cam_right/   (PlatformIO project for right camera)
```


| 4. Git Development Workflow

To prevent conflicts and ensure the project always works, we will follow a simple branching strategy.

1. The 'main' branch is ****SACRED****. It must always contain a stable, working version of the project.
2. The 'develop' branch is our main integration branch. All new features are merged here for testing.
3. When starting a new task (e.g., adding motor control), create a 'feature/' branch from 'develop'.
`git checkout -b feature/motor-control*`
4. Do all your work on your feature branch.
5. When the feature is finished, create a Pull Request to merge it into 'develop'.
6. After testing on 'develop', we merge 'develop*' into 'main' to release a new stable version.